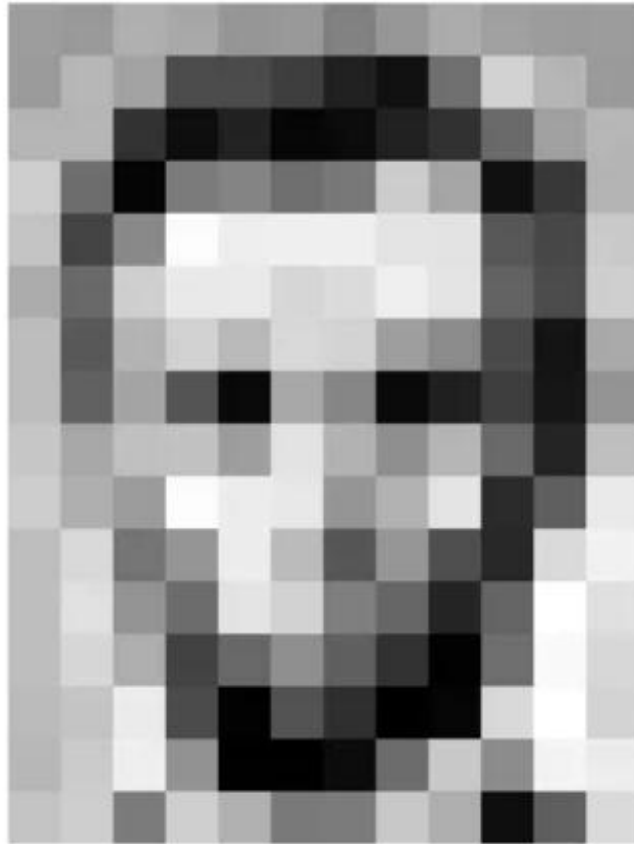


IMAGE PROCESSING AND ITS APPLICATION MODULE - 4

Prof. Nishchitha MH
Assistant Professor
Dept. of Robotics and Artificial Intelligence

Convolutional Neural Networks

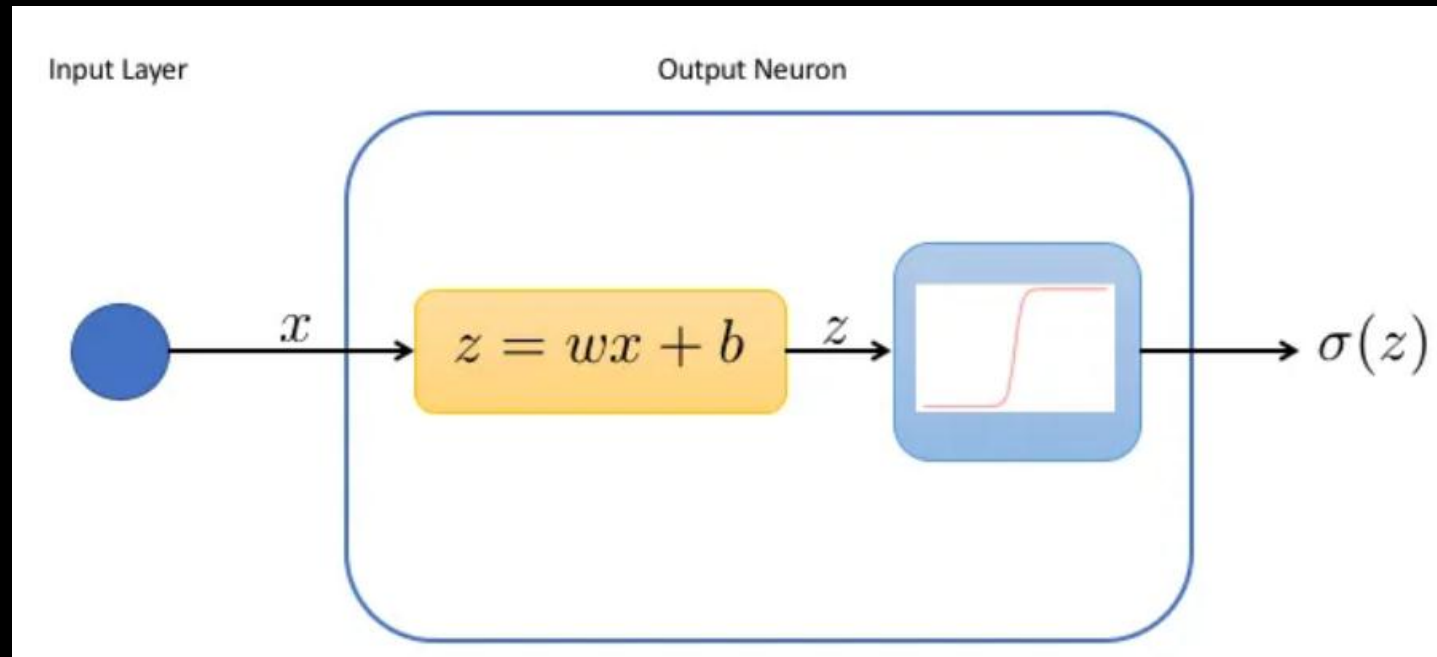


157	153	174	168	150	152	129	151	172	161	155	156
155	182	163	74	75	62	33	17	110	210	180	154
180	180	50	14	34	6	10	33	48	106	159	181
206	109	5	124	131	111	120	204	166	15	56	180
194	68	137	251	237	239	239	228	227	87	71	201
172	106	207	233	233	214	220	239	228	98	74	206
188	88	179	209	185	215	211	158	139	75	20	169
189	97	165	84	10	168	134	11	31	62	22	148
199	168	191	193	158	227	178	143	182	106	36	190
205	174	155	252	236	231	149	178	228	43	95	234
190	216	116	149	236	187	86	150	79	38	218	241
190	224	147	108	227	210	127	102	36	101	255	224
190	214	173	66	103	143	96	50	2	109	249	215
187	196	235	75	1	81	47	0	6	217	255	211
183	202	237	145	0	0	12	108	200	138	243	236
195	206	123	207	177	121	123	200	175	13	96	218

157	153	174	168	150	152	129	151	172	161	155	156
155	182	163	74	75	62	33	17	110	210	180	154
180	180	50	14	34	6	10	33	48	106	159	181
206	109	5	124	131	111	120	204	166	15	56	180
194	68	137	251	237	239	239	228	227	87	71	201
172	106	207	233	233	214	220	239	228	98	74	206
188	88	179	209	185	215	211	158	139	75	20	169
189	97	165	84	10	168	134	11	31	62	22	148
199	168	191	193	158	227	178	143	182	106	36	190
205	174	155	252	236	231	149	178	228	43	95	234
190	216	116	149	236	187	86	150	79	38	218	241
190	224	147	108	227	210	127	102	36	101	255	224
190	214	173	66	103	143	96	50	2	109	249	215
187	196	235	75	1	81	47	0	6	217	255	211
183	202	237	145	0	0	12	108	200	138	243	236
195	206	123	207	177	121	123	200	175	13	96	218

Neural Network

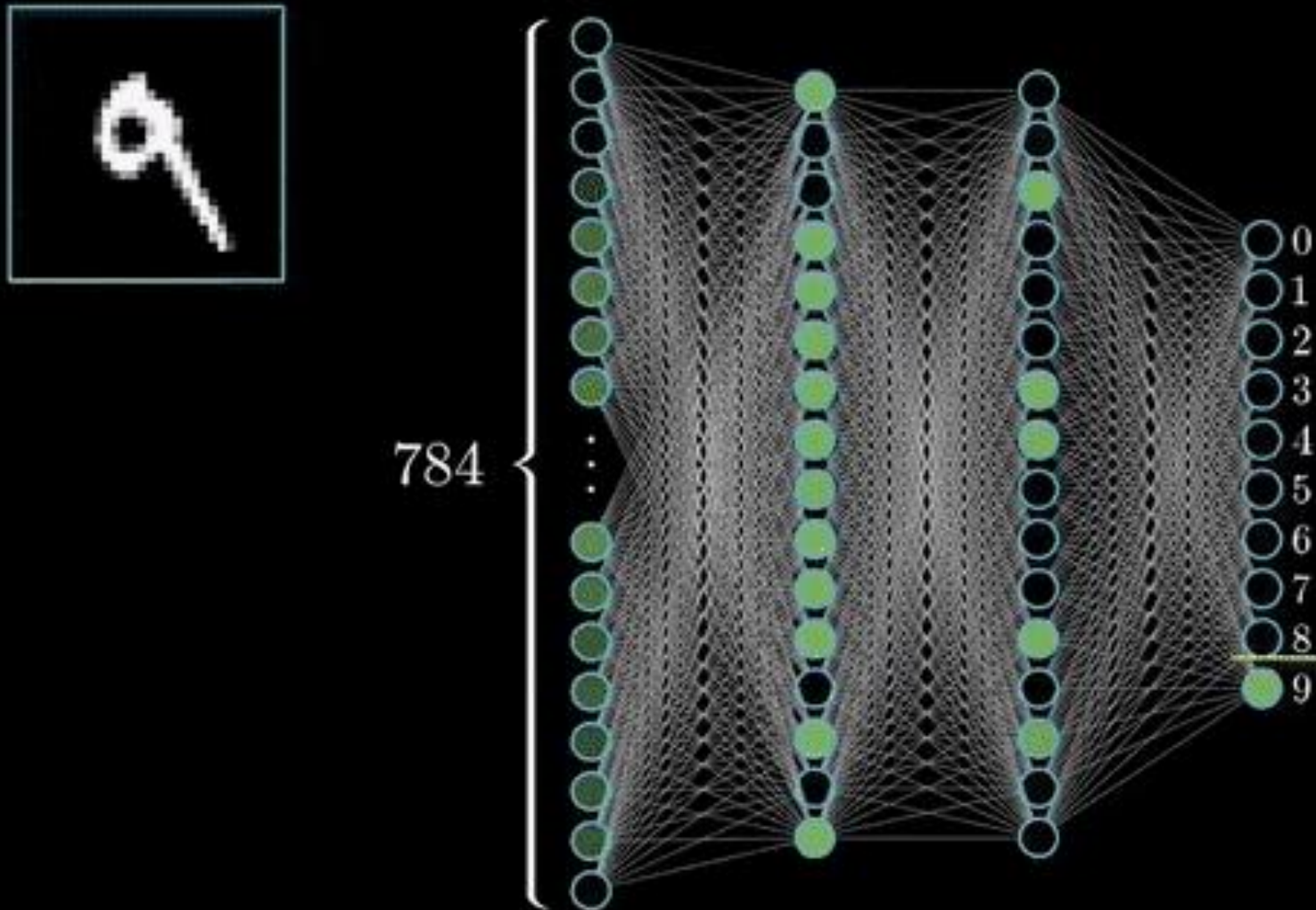
- **Neurons:** The most basic unit in a neural network. They are composed of a **sum of linear functions** and a **non-linear function** known as the **activation function** is applied to them.



Neural Network

- **Input layer:** Each neuron in the input layer corresponds to one of the input features.
- **Hidden Layer:** The layers between the input and the output layer. Each neuron in this layer is **summed** by the result of the neurons in the previous layers and multiplied by a **non-linear function**.
- **Output Layer:** The number of neurons in the output layer corresponds to the number of output classes (In case we are facing a **regression** problem the output layer will only have **one neuron**).

Neural Network



Convolutional Neural Networks

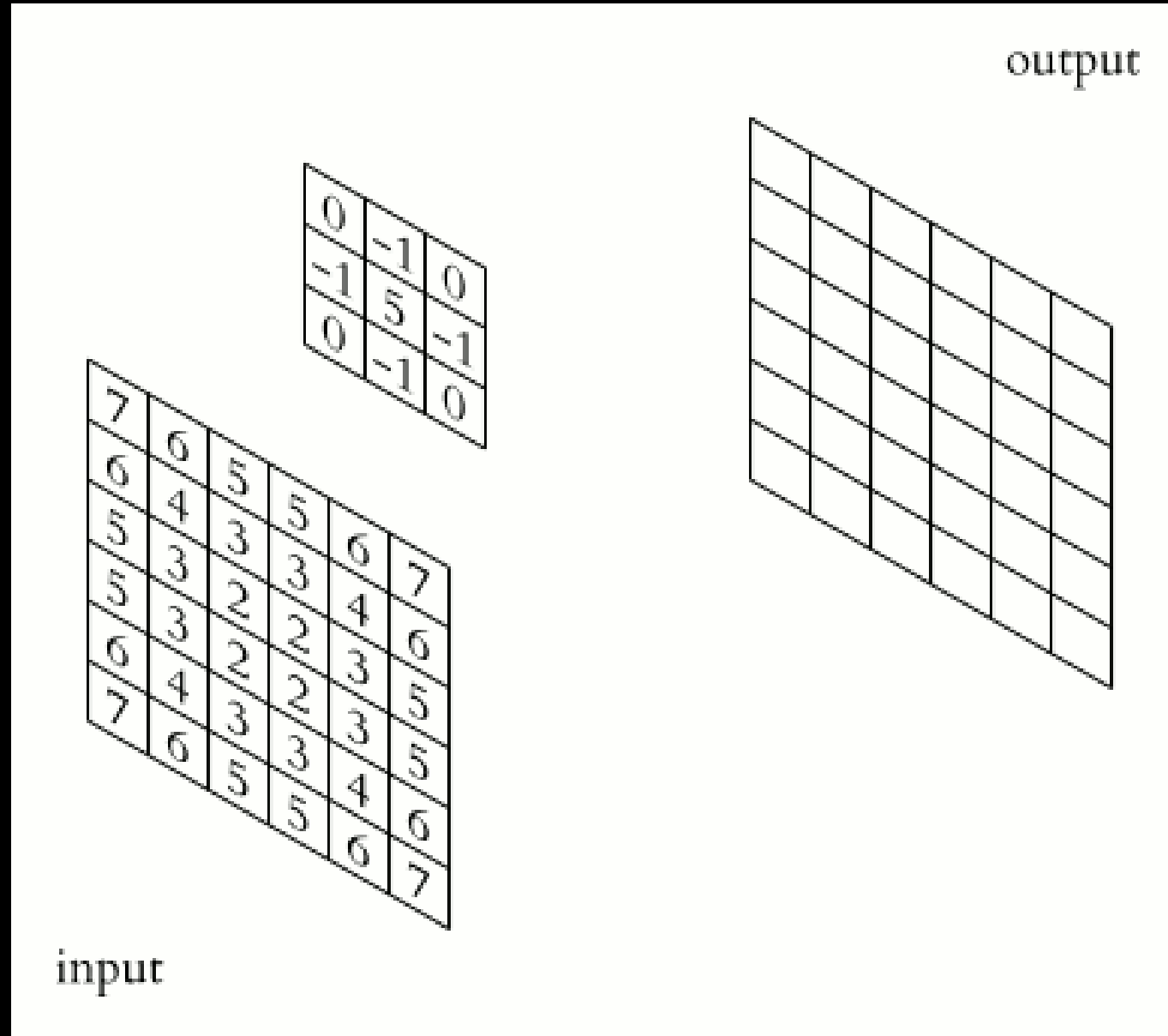
Convolutional layers

- They are the fundamental building blocks of CNNs. These layers perform a critical mathematical operation known as **convolution**.
- This process entails the application of **specialized filters known as kernels**, that traverse through the input image to learn complex visual patterns.

Kernels

They are essentially small matrices of numbers. These filters move across the image performing **element-wise multiplication** with the part of the image they cover, extracting features such as **edges, textures, and shapes**.

Kernel operation



Convolution Operation

The convolution operation involves multiplying **the kernel values** by the **original pixel values** of the image and then **summing up the results**.

Input		Kernel		Output																	
<table><tr><td>0</td><td>1</td><td>2</td></tr><tr><td>3</td><td>4</td><td>5</td></tr><tr><td>6</td><td>7</td><td>8</td></tr></table>	0	1	2	3	4	5	6	7	8	*	<table><tr><td>0</td><td>1</td></tr><tr><td>2</td><td>3</td></tr></table>	0	1	2	3	=	<table><tr><td>19</td><td>25</td></tr><tr><td>37</td><td>43</td></tr></table>	19	25	37	43
0	1	2																			
3	4	5																			
6	7	8																			
0	1																				
2	3																				
19	25																				
37	43																				

$$(0 \times 0) + (1 \times 1) + (3 \times 2) + (4 \times 3) = \mathbf{19}$$

$$(1 \times 0) + (2 \times 1) + (4 \times 2) + (5 \times 3) = \mathbf{25}$$

$$(3 \times 0) + (4 \times 1) + (6 \times 2) + (7 \times 3) = \mathbf{37}$$

$$(4 \times 0) + (5 \times 1) + (7 \times 2) + (8 \times 3) = \mathbf{43}$$

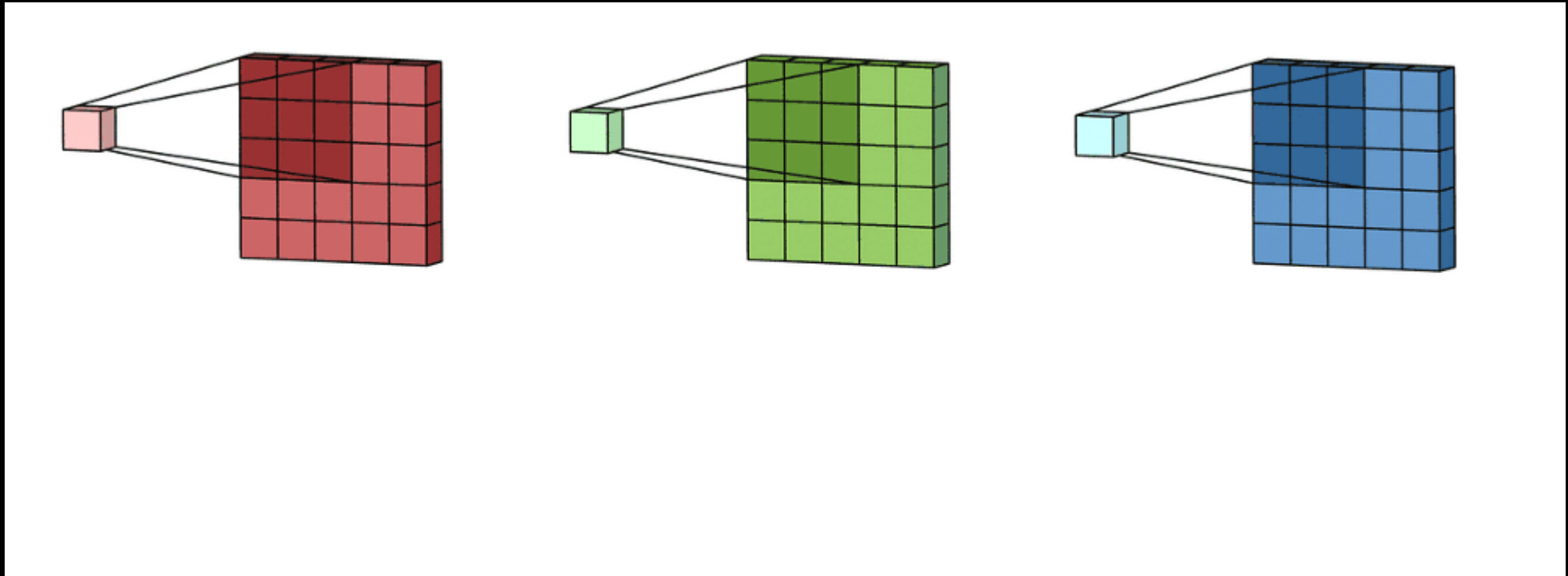
The output matrix of this process is known as the **Feature map**.

Components of convolutional layers

1. Channels
2. Stride
3. Padding

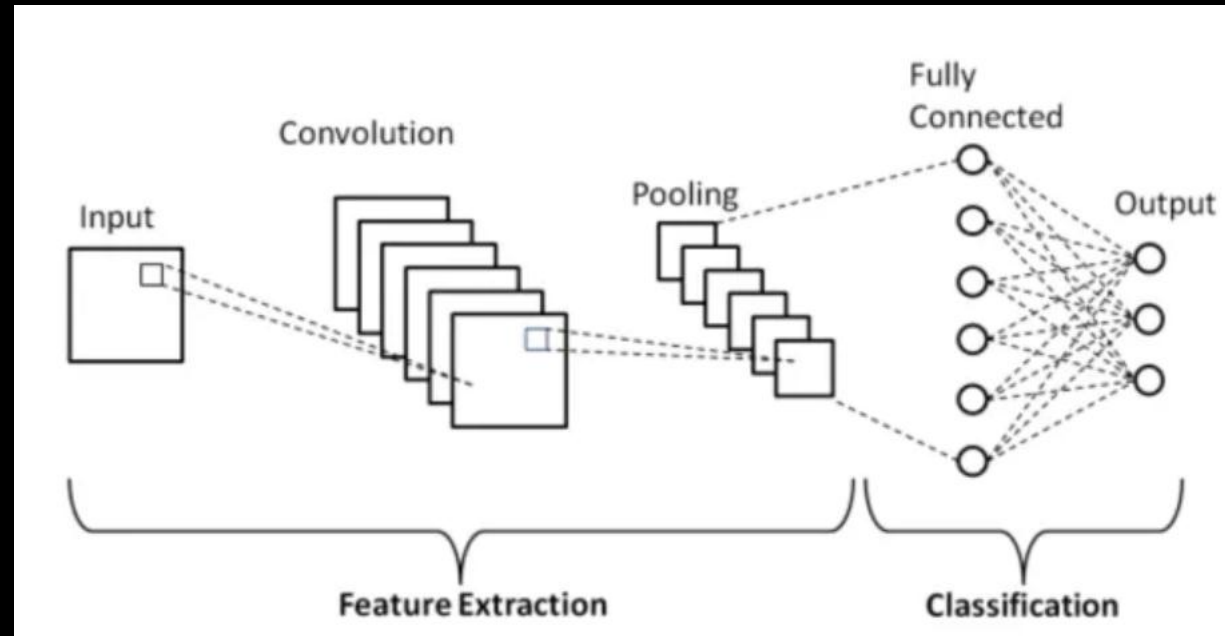
Channels

- For an RGB image, there are typically **separate kernels** for **each color channel** because different features might be more visible or relevant in one channel compared to the others.



Depth of the layer

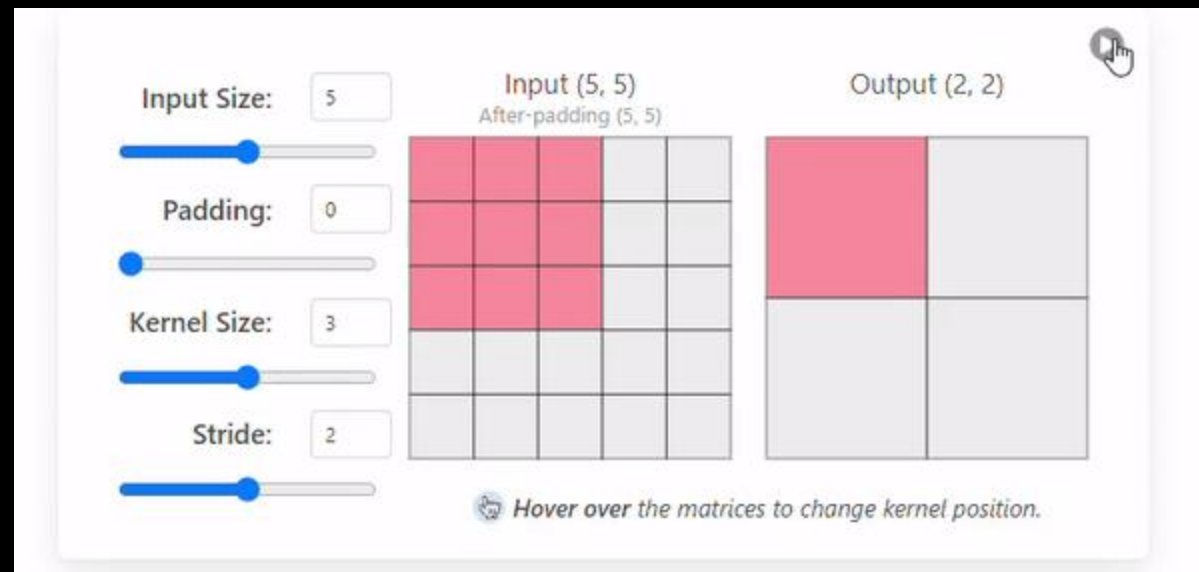
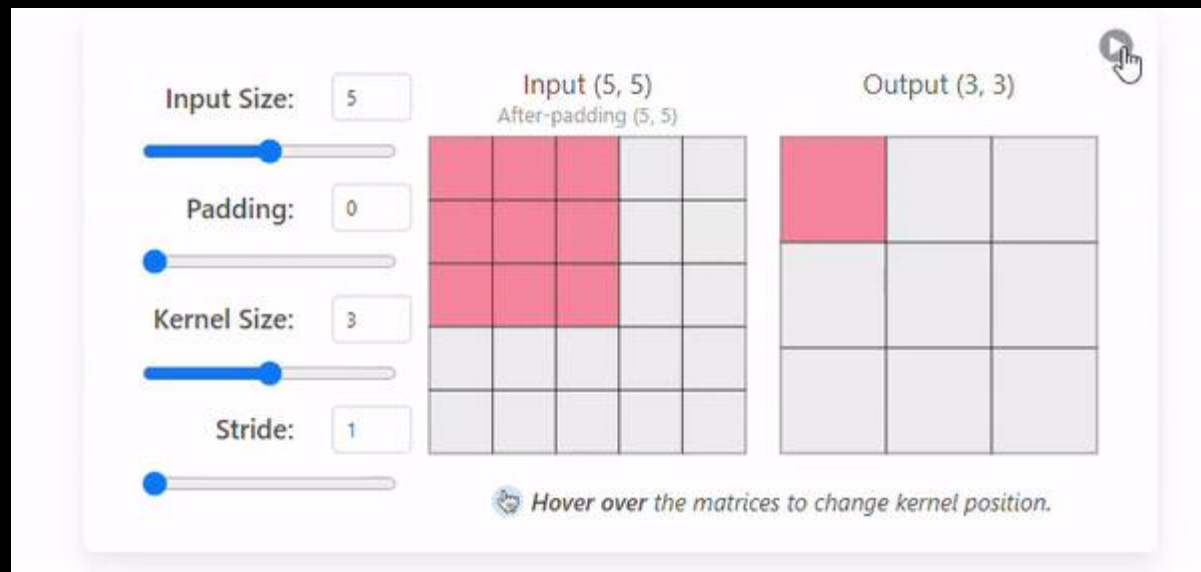
- The ‘**depth**’ of a layer refers to the number of kernels it contains. Each filter produces a separate **feature map**, and the collection of these feature maps forms the ***complete output of the layer***.
- ***For example***, let’s say that you have a gray-scale image of a cat, you could create a channel specialized in detecting the ears and another in the mouth.



Stride

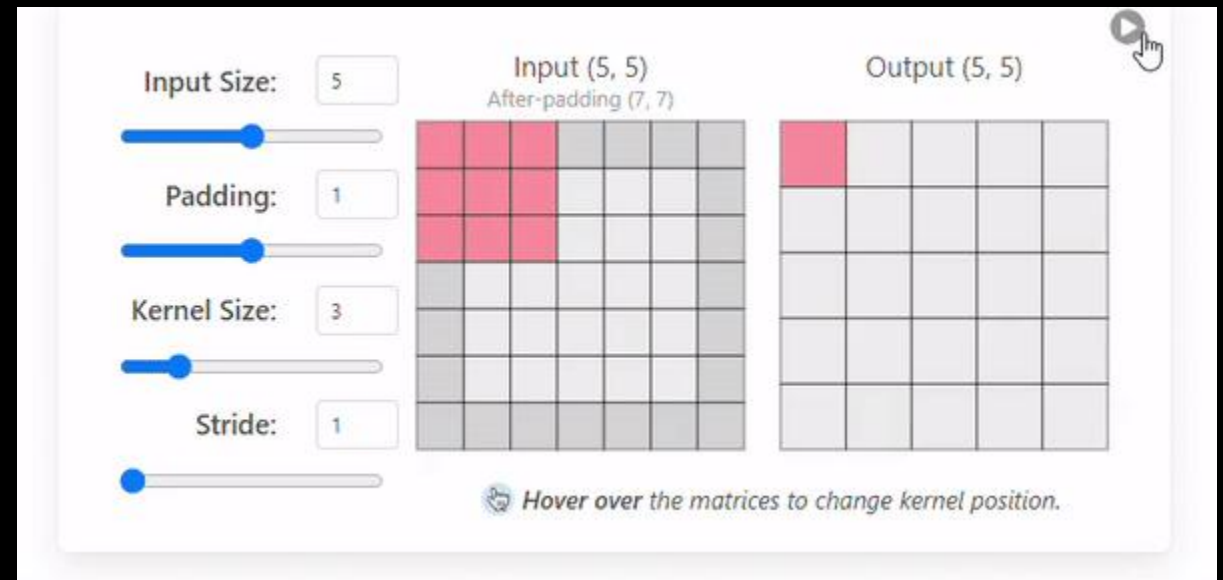
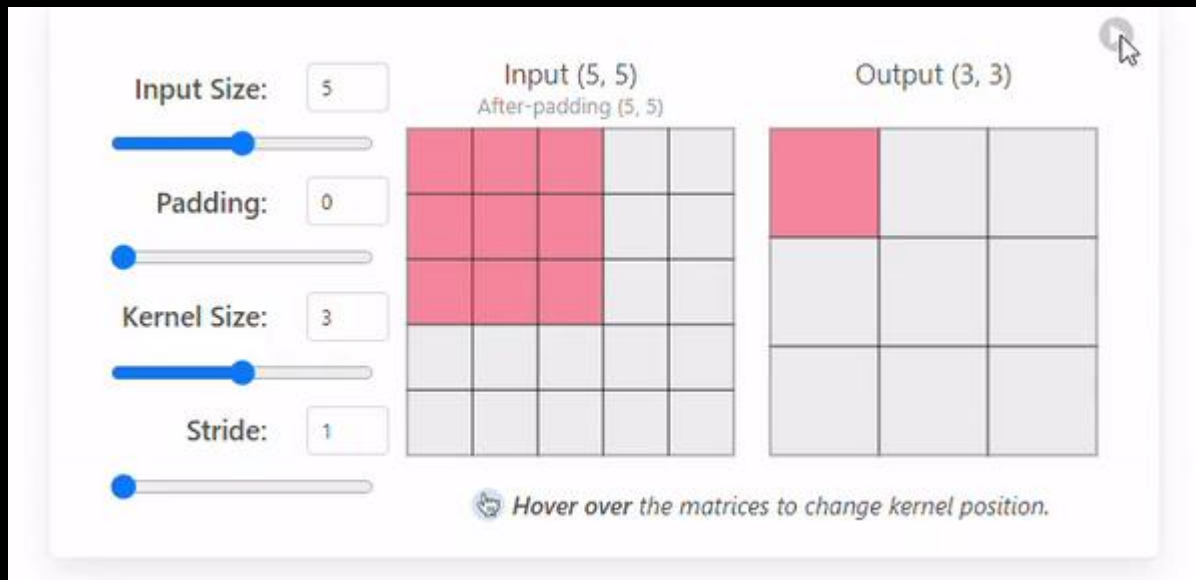
- Stride refers to the **number of pixels by which a kernel moves across the input image**.
- **Increasing the stride will allow the filter to cover a larger area of the input image**, which can be useful for capturing **more global features**.
- In contrast, **lowering the stride will capture finer and more local details**.
- In addition, increasing the stride will control **overfitting** and **reduce computational efficiency** as it will reduce the spatial dimensions of the feature map.

Stride



Padding

- Padding refers to the **addition of extra pixels around the edge** of the input image.
- The purpose of padding is to **adjust the spatial size** of the output of a convolutional operation and to **preserve spatial information at the borders**.
- Padding = 0 (focus on the edges and count how many times the kernel is passing through them)



Padding

- Padding **increases the size of the output feature map**. If you increase the padding while keeping the kernel size and stride constant, the convolution operation has more “room” to take place, **resulting in a larger output**.

- The output size of a convolutional layer can be calculated using the following formula:

$$\text{Output size} = \frac{\text{Input size} + 2 \times \text{Padding} - \text{Kernel size}}{\text{Stride}} + 1$$

- “**2 × Padding**” accounts for padding applied to both the left and right sides (or top and bottom sides) of the input.
- “**+ 1**” accounts for the initial position of the filter, which starts at the beginning of the padded input.

Pooling Layers

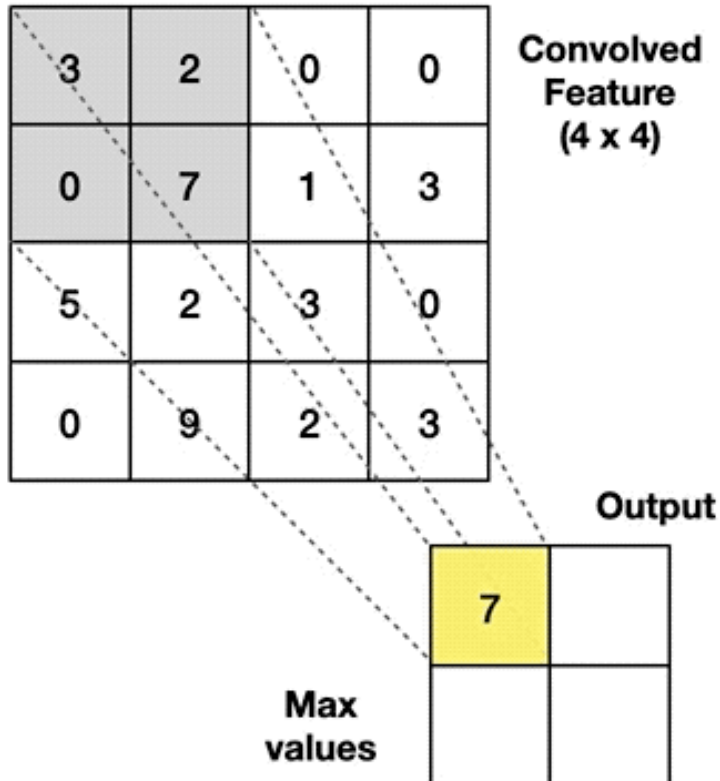
- Imagine you have a large image and want to make it smaller but keep **all the important features** like edges and colors.
- The pooling layer operates independently on every depth slice of the input. It resizes it spatially, using the **Max** or **Average** of the values in a window slid over the input data.
- In **pooling**, we are not applying any kernel to the input data, we are just **simplifying the information** with a math operation (Max or Avg).

Pooling Layers

Max Pooling

Take the **highest** value from the area covered by the kernel

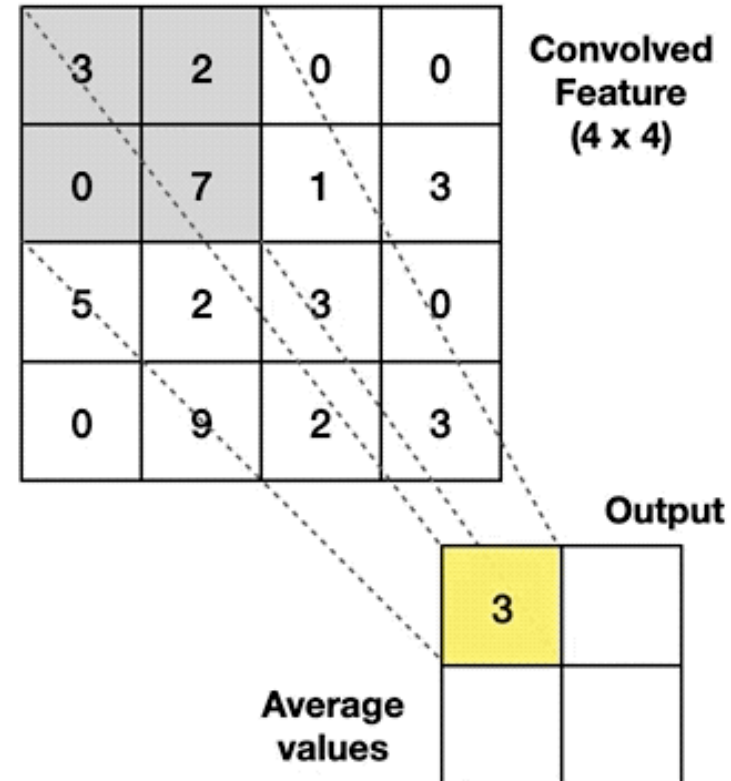
Example: Kernel of size 2 x 2; stride=(2,2)



Average Pooling

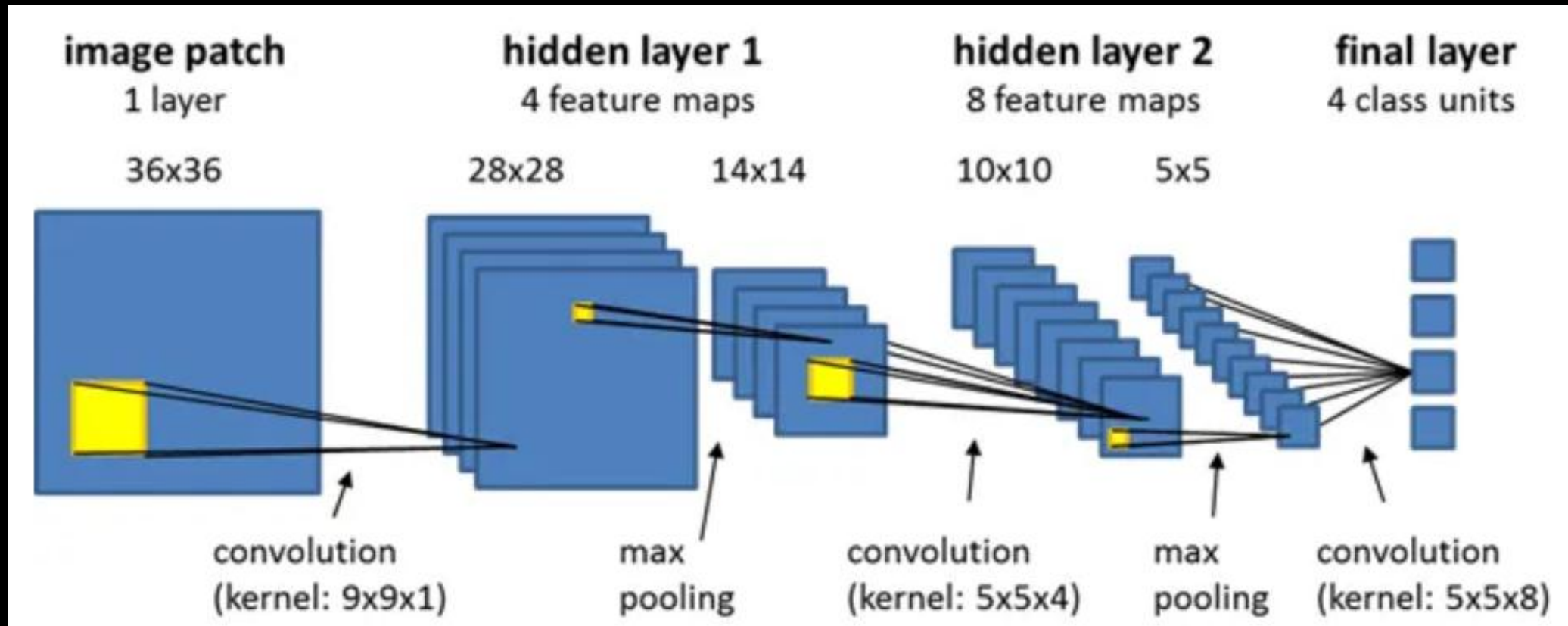
Calculate the **average** value from the area covered by the kernel

Example: Kernel of size 2 x 2; stride=(2,2)



Pooling Layers

Pooling layer is reduces the dimensions of the spatial space but it's not reducing the number of channels.



Flattening layers

1	1	0
4	2	1
0	2	1

Pooled Feature Map

Flattening



1
1
0
4
2
1
0
2
1

Why do we need flattening layers?

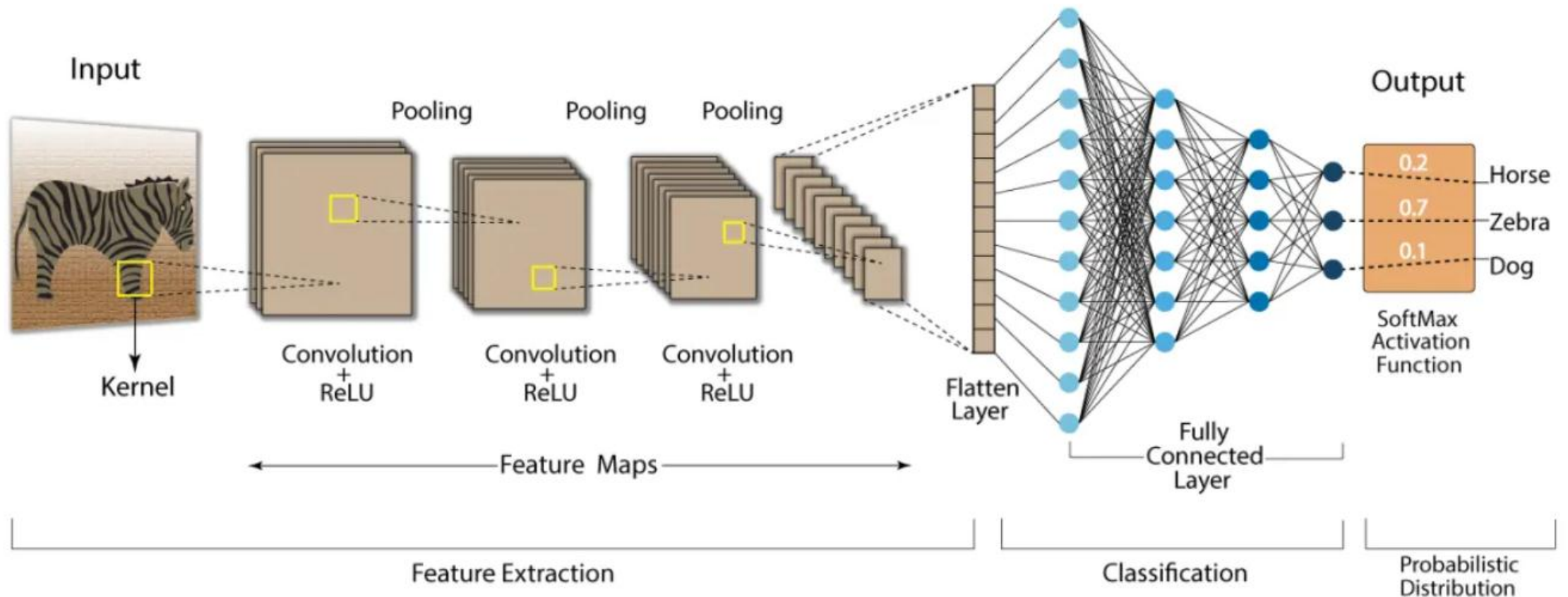
- **Integration of features**

By flattening the feature maps into a vector, the network can integrate the spatially distributed features extracted for tasks such as classification.

- **Compatibility with Dense Layers**

Fully connected layers (dense layers) are designed to operate on **1-dimensional data**, hence, flattening is a necessary step to transition from the multidimensional tensors produced by convolutional layers to the format required for dense layers.

Complete CNN representation



Sample code

```
# Convolution Layer 1
model.add(layers.Conv2D(32, (3,3), activation='relu', input_shape=(128,128,3)))
model.add(layers.MaxPooling2D(2,2))

# Convolution Layer 2
model.add(layers.Conv2D(64, (3,3), activation='relu'))
model.add(layers.MaxPooling2D(2,2))

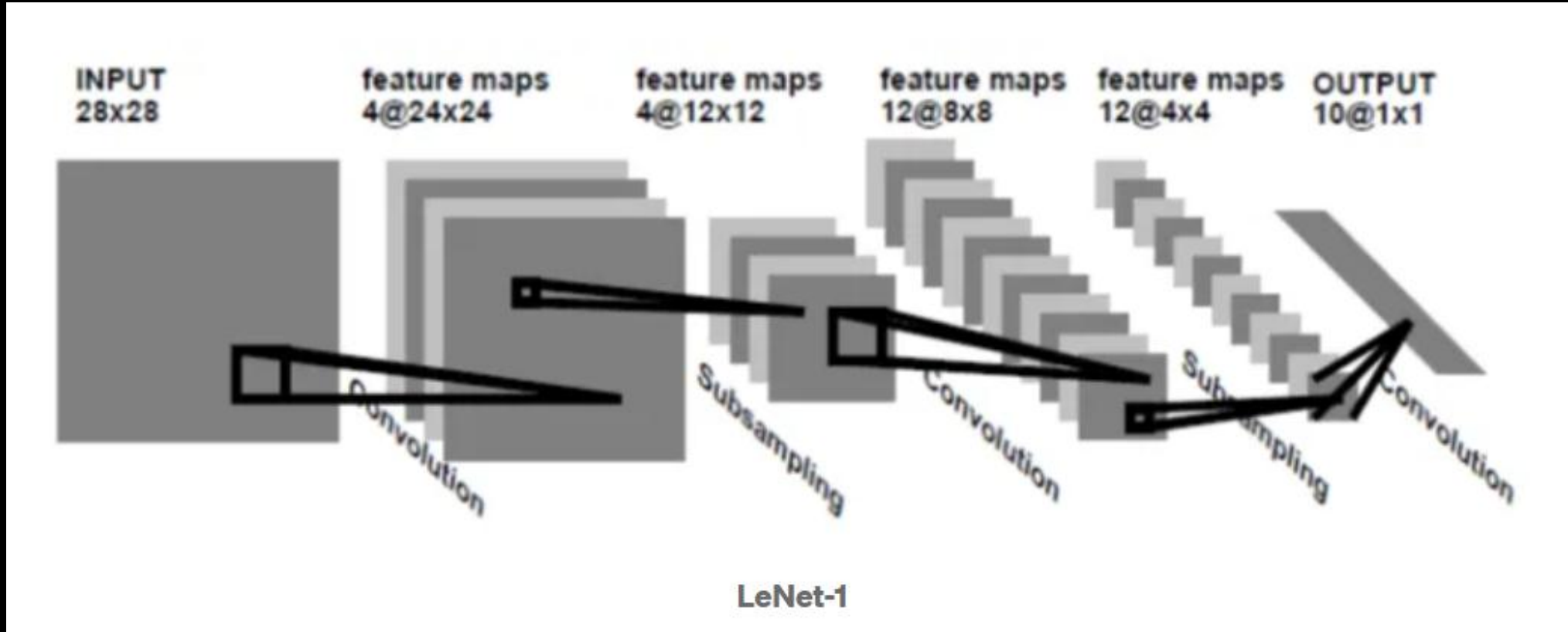
# Convolution Layer 3
model.add(layers.Conv2D(128, (3,3), activation='relu'))
model.add(layers.MaxPooling2D(2,2))

# Flatten
model.add(layers.Flatten())

# Fully Connected Layer
model.add(layers.Dense(128, activation='relu'))

# Output Layer (Binary)
model.add(layers.Dense(1, activation='sigmoid'))
```

LeNet-1



LeNet-1

- **Architecture**

- Input image
- Convolution layer
- Pooling layer
- Fully connected layer
- Output layer

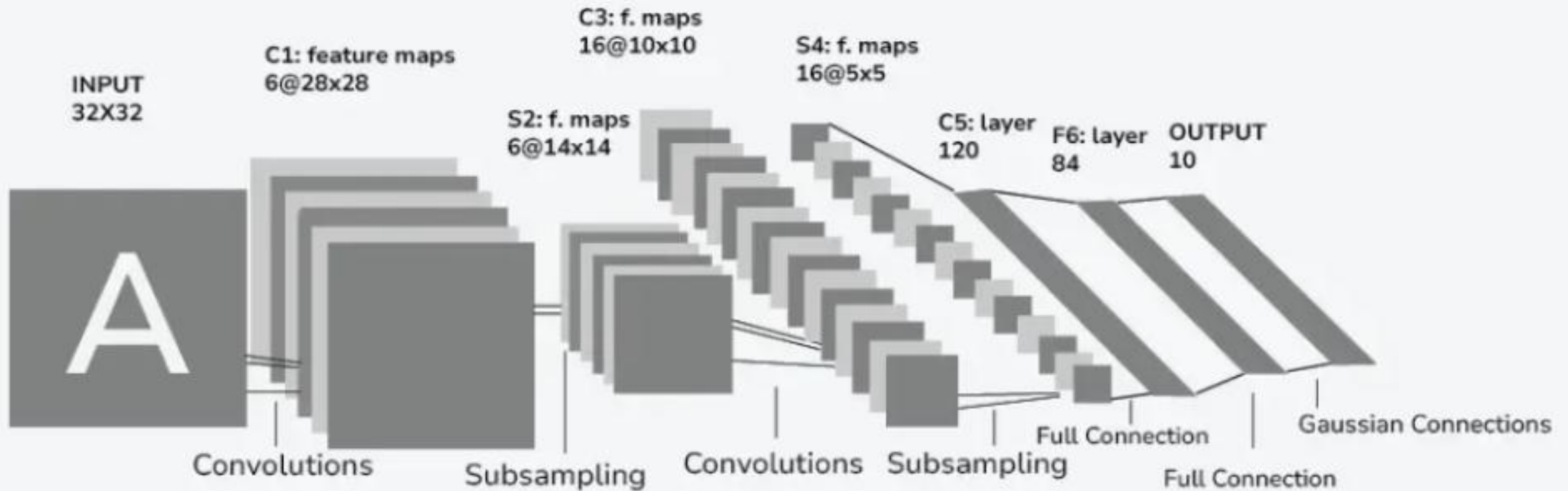
- **Characteristics**

- Very small network
- Used for basic pattern recognition
- Foundation for later LeNet models

- **Limitations**

- Low accuracy
- Small number of feature maps
- Not suitable for complex images

LeNet-5



LeNet-5

1. **Input Layer:** Accepts 32x32 pixel images, often zero-padded if original images are smaller.
2. **First Convolutional Layer (C1):** Consists of six 5x5 filters, producing six feature maps of 28x28 each.
3. **First Pooling Layer (S2):** Applies 2x2 average pooling, reducing feature maps' size to 14x14.
4. **Second Convolutional Layer (C3):** Uses sixteen 5x5 filters, but with sparse connections, outputting sixteen 10x10 feature maps.
5. **Second Pooling Layer (S4):** Further reduces feature maps to 5x5 using 2x2 average pooling.
6. **Fully Connected Layers:**
 1. **First Fully Connected Layer (C5):** Fully connected with 120 nodes.
 2. **Second Fully Connected Layer (F6):** Comprises 84 nodes.
7. **Output Layer:** Softmax or Gaussian activation that outputs probabilities across 10 classes (digits 0-9).

LeNet-5

First Convolutional Layer (C1): Consists of six 5x5 filters, producing six feature maps of 28x28 each.

- **Input Image Size:** 32×32
- **Six 5x5 Filters**
- A **filter (kernel)** is a small matrix used to scan the image and extract features like: edges, corners, textures . patterns
- Here:
 - Number of filters = 6
 - Filter size = 5×5
- So the network learns **6 different features** from the image.
- Each filter produces **one feature map**.

LeNet-5

- When a **5×5 filter** moves over a **32×32 image** without padding, the output size is calculated using:

$$\text{Output Size} = \frac{N - F}{S} + 1$$

Where:

- N = Input size = 32
- F = Filter size = 5
- S = Stride = 1

$$\begin{aligned} &= \frac{32 - 5}{1} + 1 \\ &= 27 + 1 = 28 \end{aligned}$$

- So output size = **28 × 28**

LeNet-5

- **Six Feature Maps Because:**
 - There are **6 filters**
 - Each filter produces **one feature map**
- So output: *6 feature maps of size 28×28*

<u>Feature Map</u>	<u>Detects</u>
Map 1	Vertical edges
Map 2	Horizontal edges
Map 3	Corners
Map 4	Curves
Map 5	Textures
Map 6	Patterns

LeNet-5

First Pooling Layer (S2): Applies 2x2 average pooling, reducing feature maps' size to 14x14.

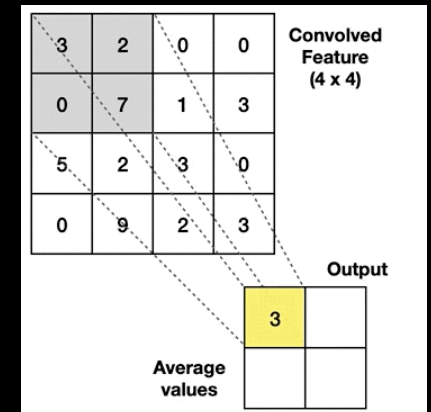
- Input to S2: $28 \times 28 \times 6$
- Average pooling output: $(2 + 4 + 6 + 8)/4 = 5$
So that 2x2 region becomes **one pixel (value = 5)**
- So, the size is reduced by half.

$$\begin{aligned} \text{Output Size} &= \frac{\text{Input Size}}{2} \\ &= \frac{28}{2} = 14 \end{aligned}$$

What is Pooling?

Pooling is used to:

- Reduce image size
- Reduce computation
- Remove noise
- Keep important features
- Prevent overfitting
- Pooling summarizes a small region into one value.



LeNet-5

Second Convolutional Layer (C3): Uses sixteen 5x5 filters, but with sparse connections, outputting sixteen 10x10 feature maps.

- So input to C3: $14 \times 14 \times 6$
- 3 layer uses: **16 filters**, Each filter size = 5×5 , Stride = 1 ,No padding
 - Each filter produces **one feature map**.
- So output feature maps = **16**

$$Output = \frac{N - F}{S} + 1$$

$$Output = (14 - 5) + 1 = 10$$

- Final output of C3: $10 \times 10 \times 16$

LeNet-5

Sparse Connections

- In normal convolution:
- Each filter connects to **all previous feature maps**
- But in **LeNet-5 C3 layer**:
- Each filter connects to **only some of the previous 6 feature maps**
- Not all connections are used
- This is called **sparse connectivity**
 - *Reduce number of parameters*
 - *Force network to learn different features*
 - *Reduce computation*
 - *Avoid overfitting*

LeNet-5

- So instead of fully connecting: $6 \rightarrow 16$
- They used **partial connections**.

Example:

- Filter 1 connects to maps 1,2,3
- Filter 2 connects to maps 2,3,4
- Filter 3 connects to maps 3,4,5

This creates **different feature combinations**.

LeNet-5

Second Pooling Layer (S4): Further reduces feature maps to 5x5 using 2x2 average pooling.

Fully Connected Layers:

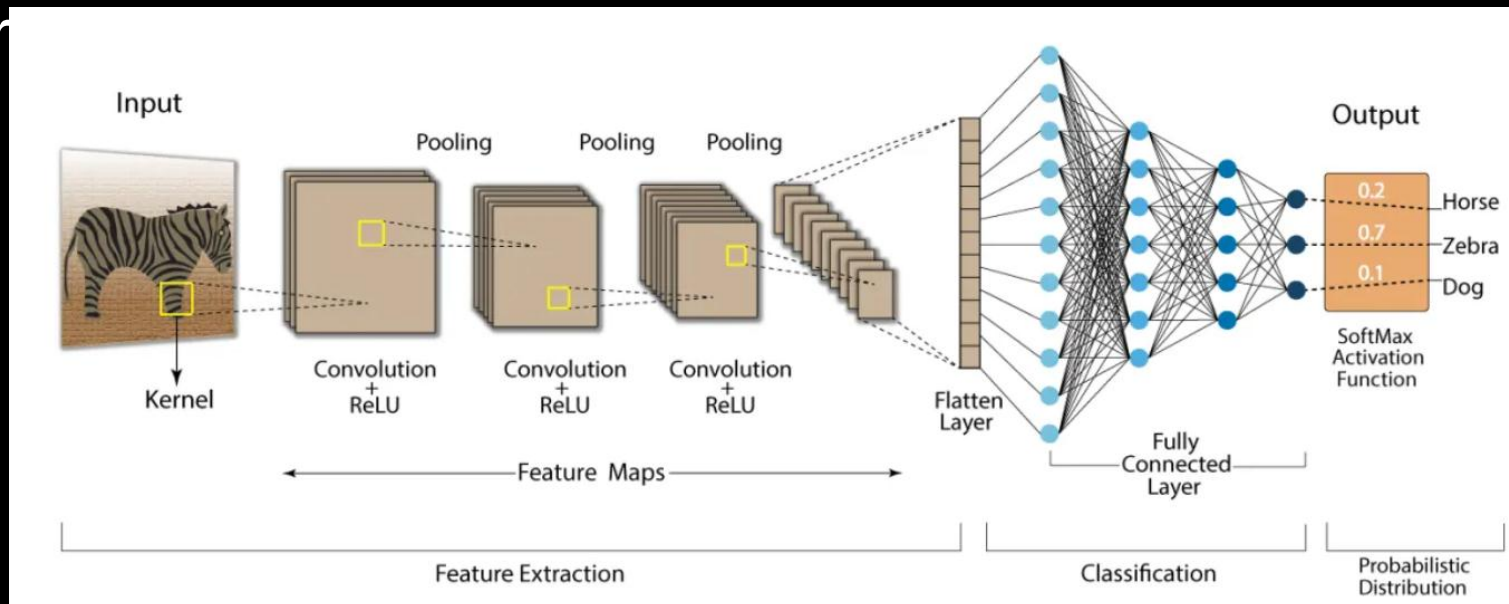
1. **First Fully Connected Layer (C5):** Fully connected with 120 nodes.
2. **Second Fully Connected Layer (F6):** Comprises 84 nodes.

LeNet-5

First Fully Connected Layer (C5): Fully connected with 120 nodes.

Before C5, the previous layer (S4) output size is: $5 \times 5 \times 16$
(16 feature maps, Each of size 5×5)

Total neur



LeNet-5

- Input neurons = 400
- Output neurons = 120
- Each of the 120 neurons connects to **all 400 inputs**
- So this layer converts: $5 \times 5 \times 16 \rightarrow 120$
- **Purpose**
 - Combines all extracted features
 - Detects high-level patterns
 - Prepares data for classification

LeNet-5

Second Fully Connected Layer (F6) – 84 Nodes

Input = 120 neurons

Output = 84 neurons

Every neuron connects to all previous neurons : $120 \rightarrow 84$

Purpose

- Further processes features
- Performs feature combination
- Prepares final classification

Historically, **84 neurons** were chosen by **Yann LeCun** based on experiments and performance for digit recognition tasks.

It helps:

- Reduce dimensions
- Avoid overfitting
- Improve classification accuracy

LeNet-5

Final Output Layer

- After F6: Final layer has **10 neurons**
- Represents digits **0–9**
- Uses **Softmax classifier**

84 → 10

LeNet-5

Layer	Operation	Output Size
Input	Image	32×32
C1	Conv	$28 \times 28 \times 6$
S2	Avg Pool	$14 \times 14 \times 6$
C3	Conv	$10 \times 10 \times 16$
S4	Avg Pool	$5 \times 5 \times 16$
C5	Fully Connected	120
F6	Fully Connected	84
Output	Softmax	10

LeNet-5

Layer	What Happens
C1	Detect edges
S2	Reduce size
C3	Detect complex shapes
S4	Reduce size
C5	Detect full objects
FC	Classification

Layer	Function
Conv Layers	Feature extraction
Pooling Layers	Size reduction
Fully Connected	Classification
Output	Final prediction

AlexNet

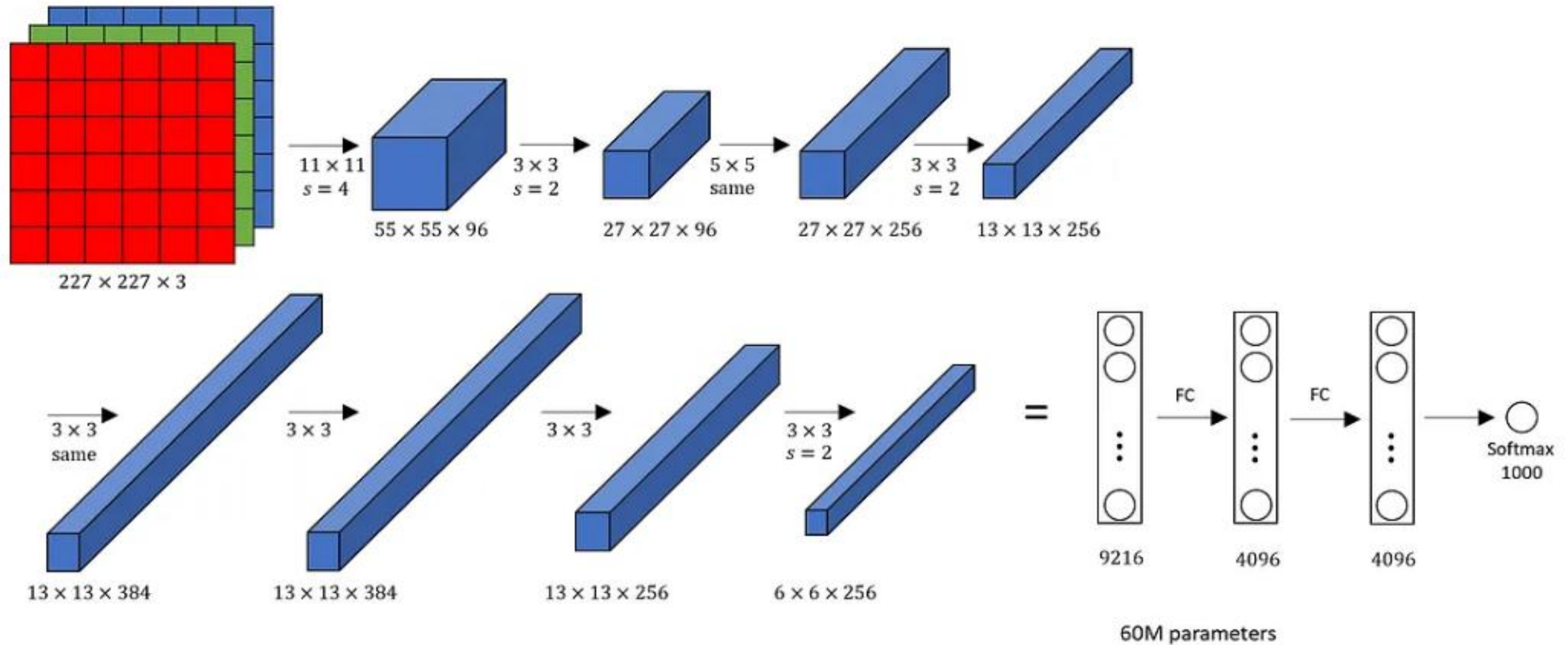
Traditional machine learning used manual feature extraction,
Accuracy was low

AlexNet introduced:

- Deep CNN
- ReLU activation
- Dropout
- GPU training
- Data augmentation

This drastically improved image classification accuracy.

AlexNet



AlexNet consists of 5 convolution layers, 3 max-pooling layers, 2 Normalized layers, 2 fully connected layers and 1 SoftMax layer.

AlexNet

Input Image Size: $27 \times 227 \times 3$ (Height $\times$ Width $\times$ Channels)

Conv1 Layer

Filters = 96

Filter size = 11×11

Stride = 4

Padding = 0

Input = 227×227

$$\begin{aligned} \text{Output} &= \frac{227 - 11}{4} + 1 \\ &= \frac{216}{4} + 1 \\ &= 54 + 1 = 55 \end{aligned}$$

So output:

$$55 \times 55 \times 96$$

Max Pooling 1

Pool size = 3×3

Stride = 2

$$\begin{aligned} \text{Output} &= \frac{55 - 3}{2} + 1 \\ &= \frac{52}{2} + 1 \\ &= 26 + 1 = 27 \end{aligned}$$

Output:

$$27 \times 27 \times 96$$

AlexNet

Conv2 Layer

Filters = 256

Filter size = 5×5

Padding = 2

Stride = 1

Input = 27

$$\begin{aligned} \text{Output} &= \frac{27 - 5 + 2(2)}{1} + 1 \\ &= \frac{27 - 5 + 4}{1} + 1 \\ &= 26 + 1 = 27 \end{aligned}$$

Output:

$$27 \times 27 \times 256$$

Max Pooling 2

Pool = 3×3

Stride = 2

$$\begin{aligned} \text{Output} &= \frac{27 - 3}{2} + 1 \\ &= \frac{24}{2} + 1 \\ &= 12 + 1 = 13 \end{aligned}$$

Output:

$$13 \times 13 \times 256$$

AlexNet

Conv3 Layer

Filters = 384

Filter = 3×3

Padding = 1

Stride = 1

$$\begin{aligned} \text{Output} &= \frac{13 - 3 + 2}{1} + 1 \\ &= 12 + 1 = 13 \end{aligned}$$

Output:

13 × 13 × 384

Conv4 Layer Same settings as
Conv3: 13 × 13 × 384

Conv5 Layer

Filters = 256

Filter = 3×3

Padding = 1

Output:

13 × 13 × 256

AlexNet

Max Pooling 3

Pool = 3×3

Stride = 2

$$\begin{aligned} \text{Output} &= \frac{13 - 3}{2} + 1 \\ &= \frac{10}{2} + 1 \\ &= 5 + 1 = 6 \end{aligned}$$

Output:

$$6 \times 6 \times 256$$

Flatten Layer

Convert to 1D vector:

$$6 \times 6 \times 256 = 9216$$

So input to Fully Connected layer = **9216 neurons**

FC6: 9216 → 4096

FC7: 4096 → 4096

FC8 (Output Layer): 4096 → 1000

AlexNet

Layer		Feature Map	Size	Kernel Size	Stride	Activation
Input	Image	1	227x227x3	-	-	-
1	Convolution	96	55 x 55 x 96	11x11	4	relu
	Max Pooling	96	27 x 27 x 96	3x3	2	relu
2	Convolution	256	27 x 27 x 256	5x5	1	relu
	Max Pooling	256	13 x 13 x 256	3x3	2	relu
3	Convolution	384	13 x 13 x 384	3x3	1	relu
4	Convolution	384	13 x 13 x 384	3x3	1	relu
5	Convolution	256	13 x 13 x 256	3x3	1	relu
	Max Pooling	256	6 x 6 x 256	3x3	2	relu
6	FC	-	9216	-	-	relu
7	FC	-	4096	-	-	relu
8	FC	-	4096	-	-	relu
Output	FC	-	1000	-	-	Softmax

AlexNet

Layer	Type	Description
Input	Image	$227 \times 227 \times 3$
Conv1	Convolution	96 filters, 11×11
Pool1	Max Pooling	3×3
Conv2	Convolution	256 filters, 5×5
Pool2	Max Pooling	3×3
Conv3	Convolution	384 filters
Conv4	Convolution	384 filters
Conv5	Convolution	256 filters
Pool3	Max Pooling	3×3
FC6	Fully Connected	4096 neurons
FC7	Fully Connected	4096 neurons
FC8	Fully Connected	1000 neurons
Output	Softmax	Classification

Key Features of AlexNet

ReLU Activation Function

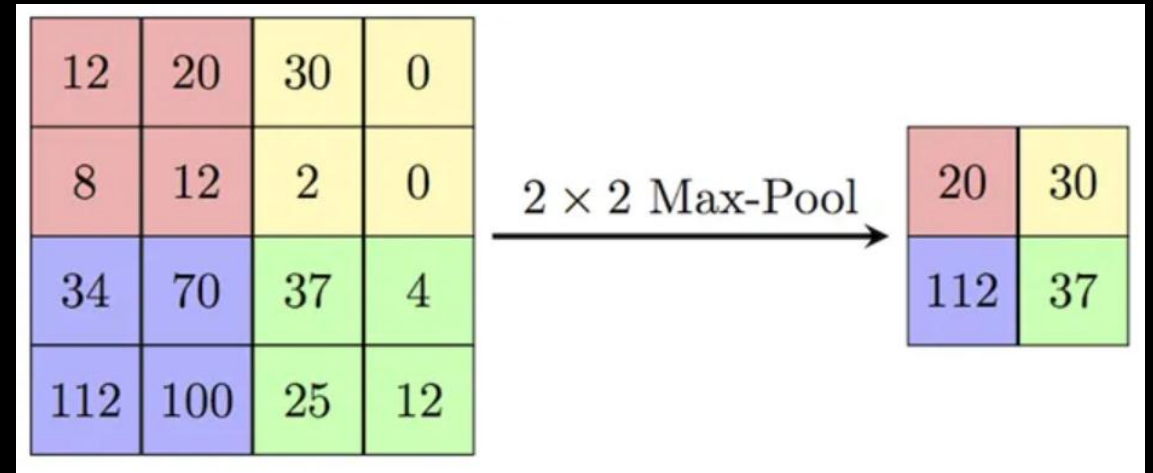
AlexNet used **ReLU** instead of sigmoid/tanh.

$$f(x) = \max(0, x)$$

Advantages:

- Faster training
- Avoids vanishing gradient
- Computationally efficient

Max Pooling: Used to reduce feature map size and keep important features.



AlexNet

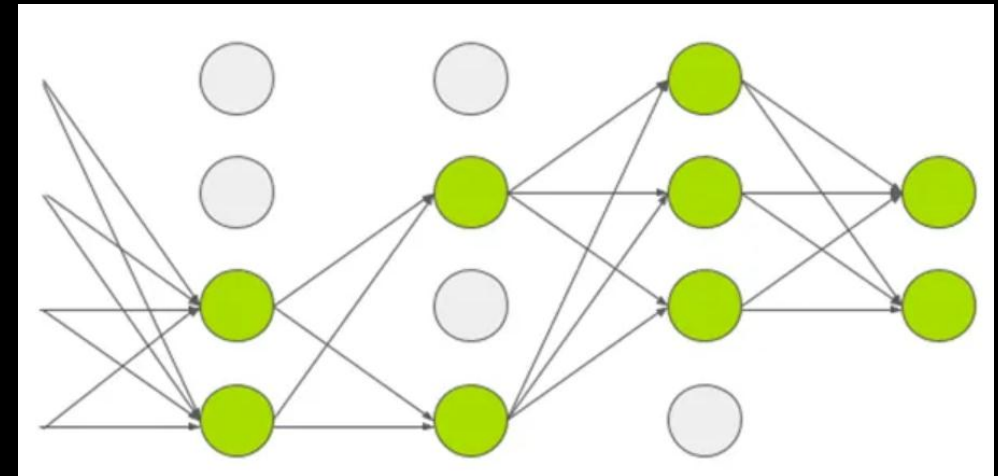
Data Augmentation

Training data was increased by:

- Rotation
- Flipping
- Cropping
- Scaling

This improved accuracy.

Dropout Layer: Dropout randomly disables neurons during training to prevent **overfitting**.



AlexNet

Advantages

- High accuracy
- Deep architecture
- Uses ReLU
- Uses Dropout
- Works well for large datasets
- GPU acceleration

Disadvantages

- Large memory requirement
- Many parameters
- Slow without GPU
- Overfitting if no dropout

VGG Neural Network – VGG-16

VGGNet is a deep Convolutional Neural Network developed by the **Visual Geometry Group**.

VGG-16 is the most popular version used for image classification

Before VGG: Networks used large filters like 11×11 , 5×5 (e.g., **AlexNet**)

VGG introduced:

- Only **3×3 convolution filters**
- Very deep architecture
- Better feature extraction
- Improved accuracy

VGG-16 has **16 weight layers**:

13 Convolution layers

3 Fully Connected layers

$$13 + 3 = 16$$

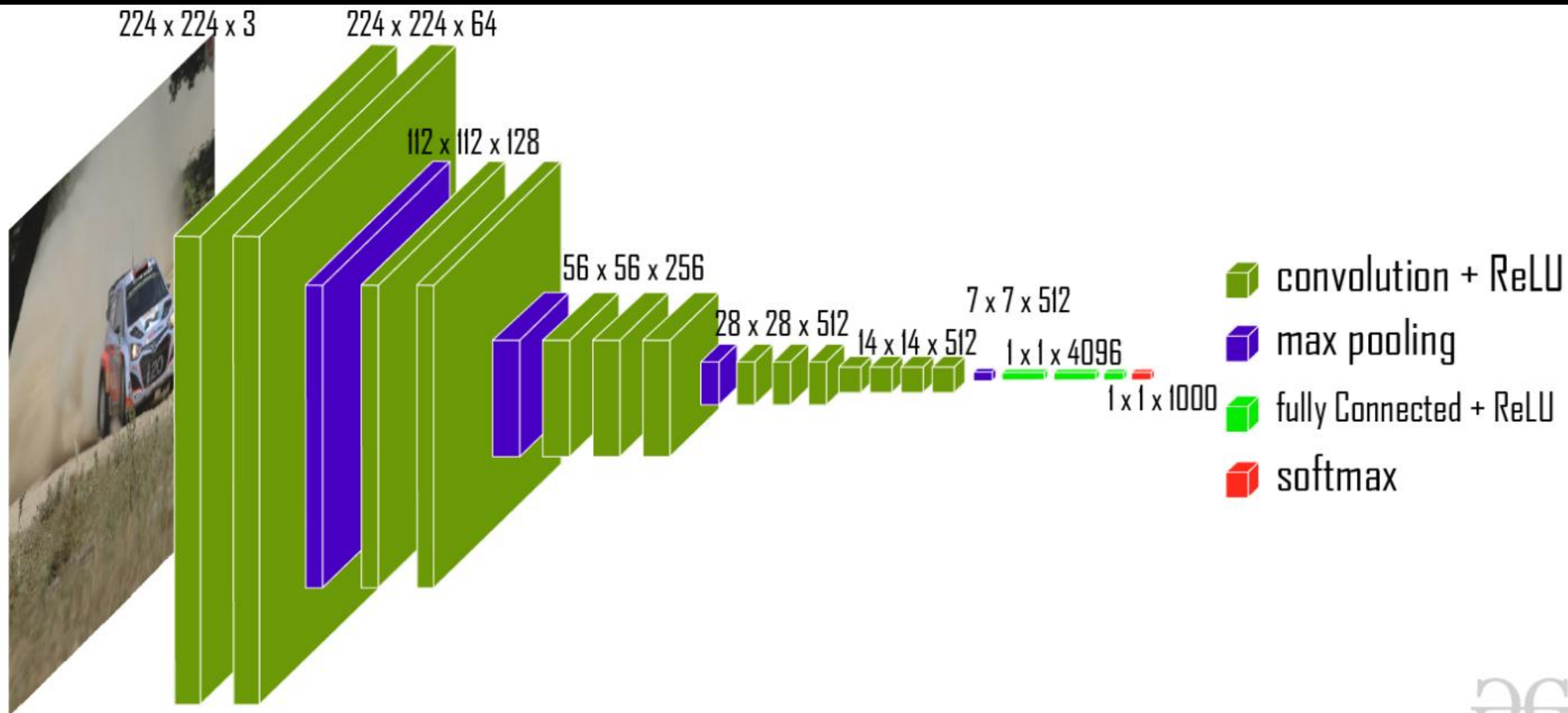
Key Idea:

Instead of one 5×5 filter $\rightarrow$ use two 3×3 filters

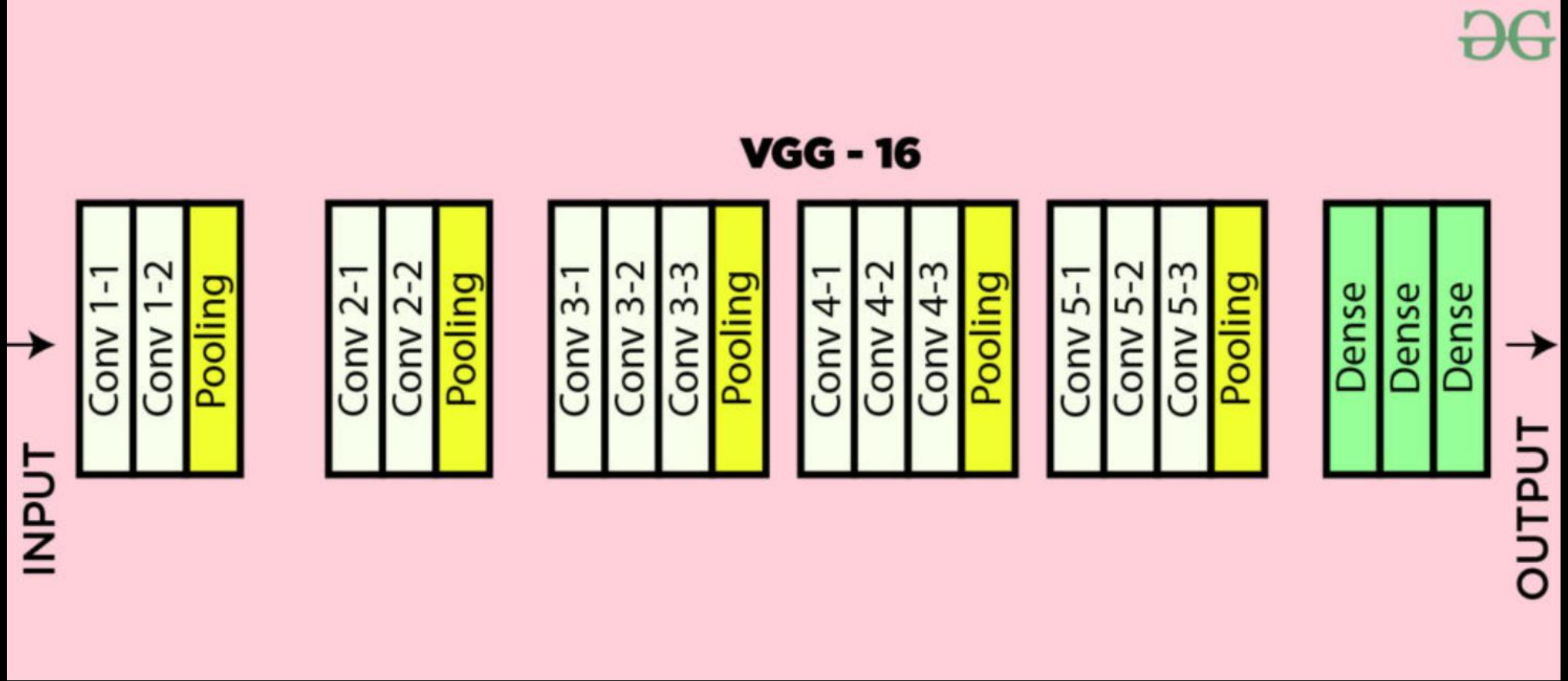
Instead of one 7×7 filter $\rightarrow$ use three 3×3 filters

This reduces parameters and increases non-linearity.

VGG Neural Network – VGG-16



VGG Neural Network – VGG-16



VGG Neural Network – VGG-16

Block	Layers	Output Size
Input	Image	224×224×3
Conv Block 1	2 Conv (64)	224×224×64
Pool	Max Pool	112×112×64
Conv Block 2	2 Conv (128)	112×112×128
Pool	Max Pool	56×56×128
Conv Block 3	3 Conv (256)	56×56×256
Pool	Max Pool	28×28×256
Conv Block 4	3 Conv (512)	28×28×512
Pool	Max Pool	14×14×512
Conv Block 5	3 Conv (512)	14×14×512
Pool	Max Pool	7×7×512
Flatten	-	25088
FC1	Dense	4096
FC2	Dense	4096
FC3	Output	1000

VGG Neural Network – VGG-16

1. Input Layer:

Input dimensions: (224, 224, 3)

2. Convolutional Layers (64 filters, 3x3 filters, same padding):

Two consecutive convolutional layers with 64 filters each and a filter size of 3x3.

Same padding is applied to maintain spatial dimensions.

3. Max Pooling Layer (2x2, stride 2):

Max-pooling layer with a pool size of 2x2 and a stride of 2.

VGG Neural Network – VGG-16

4. Convolutional Layers (128 filters, 3x3 filters, same padding):

Two consecutive convolutional layers with 128 filters each and a filter size of 3x3.

5. Max Pooling Layer (2x2, stride 2):

Max-pooling layer with a pool size of 2x2 and a stride of 2.

6. Convolutional Layers (256 filters, 3x3 filters, same padding):

Two consecutive convolutional layers with 256 filters each and a filter size of 3x3.

VGG Neural Network – VGG-16

7. Convolutional Layers (512 filters, 3x3 filters, same padding):

Two sets of three consecutive convolutional layers with 512 filters each and a filter size of 3x3.

8. Max Pooling Layer (2x2, stride 2):

Max-pooling layer with a pool size of 2x2 and a stride of 2.

9. Stack of Convolutional Layers and Max Pooling:

Two additional convolutional layers after the previous stack.
Filter size: 3x3.

10. Flattening:

Flatten the output feature map (7x7x512) into a vector of size 25088.

VGG Neural Network – VGG-16

11. Fully Connected Layers:

- Three fully connected layers with ReLU activation.
- First layer with input size 25088 and output size 4096.
- Second layer with input size 4096 and output size 4096.
- Third layer with input size 4096 and output size 1000, corresponding to the 1000 classes in the ILSVRC challenge.
- Softmax activation is applied to the output of the third fully connected layer for classification.

This architecture follows the specifications provided, including the use of ReLU activation function and the final fully connected layer outputting probabilities for 1000 classes using softmax activation.

VGG Neural Network – VGG-16

Advantages of VGG-16

- Simple architecture
- Good feature extraction
- High accuracy
- Uses small filters
- Deep network

Disadvantages

- Very large model
- High memory usage
- Slow training
- Many parameters (~138 million)

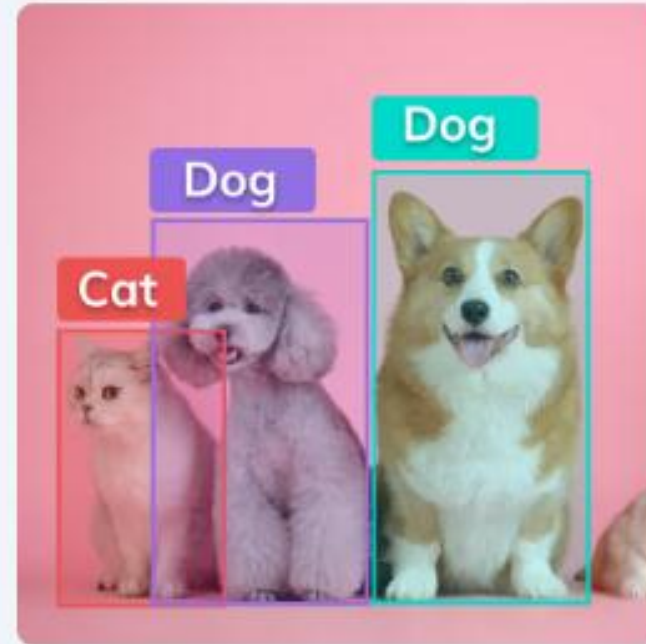
Object détection vs. image classification

Classification



Cat

Detection



Cat, Dog, Dog

Object détection vs image segmentation

- Image segmentation is the process of defining which pixels of an object class are found in an image.
- Semantic image segmentation will mark all pixels belonging to that tag, but won't define the boundaries of each object.
- Object detection instead will not segment the object, but will clearly define the location of each individual object instance with a box.
- Combining semantic segmentation with object detection leads to instance segmentation, which first detects the object instances, and then segments each within the detected boxes

Object détection vs image segmentation

**Object Detection + Semantic Segmentation
= Instance Segmentation**



Object detection

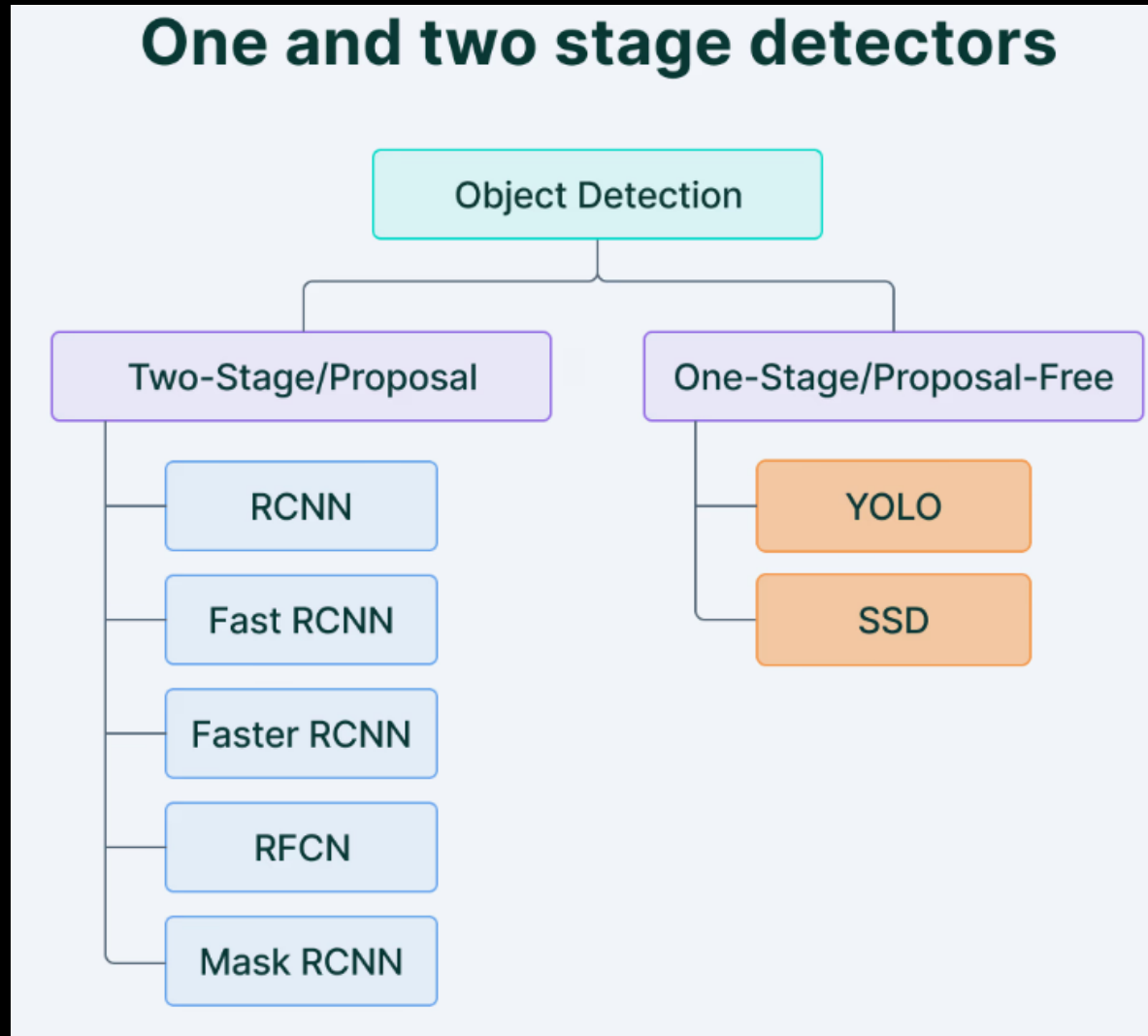


Semantic Segmentation



Instance Segmentation

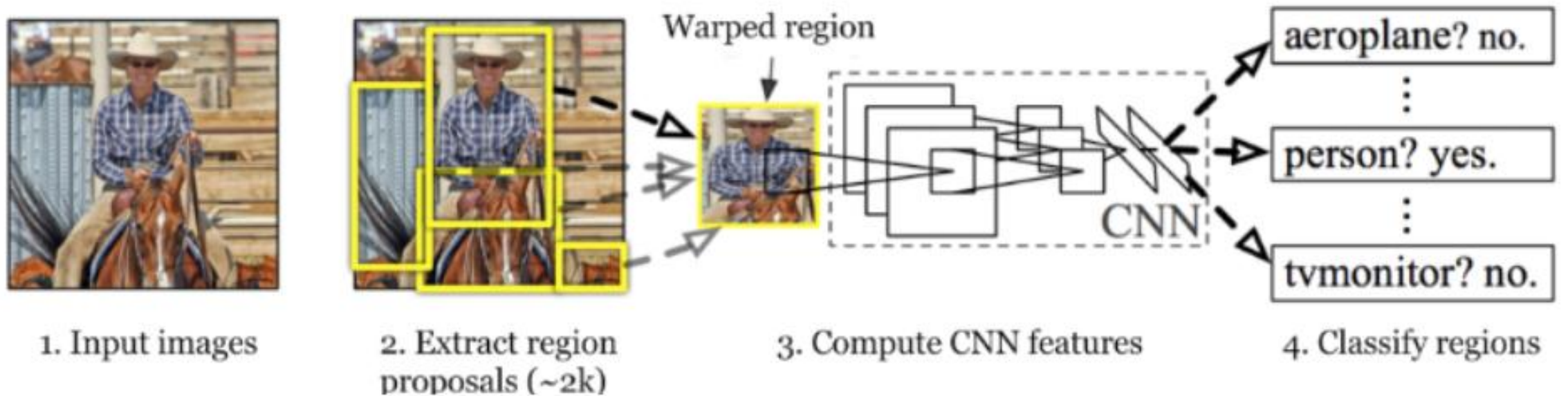
Object détection



R-CNN - Region-Based Convolutional Neural Networks

- Traditional Convolutional Neural Networks (CNNs) with fully connected layers often struggle with object detection tasks, especially when dealing with multiple objects of varying sizes and positions within an image.
- To overcome these challenges, R-CNN (Regions with CNN features) was introduced. R-CNN presents a smarter approach by using a **selective search algorithm** to generate around 2,000 region proposals from an image.
- These proposals are likely to contain objects and are individually processed to detect and localize them more efficiently.

R-CNN - Region-Based Convolutional Neural Networks

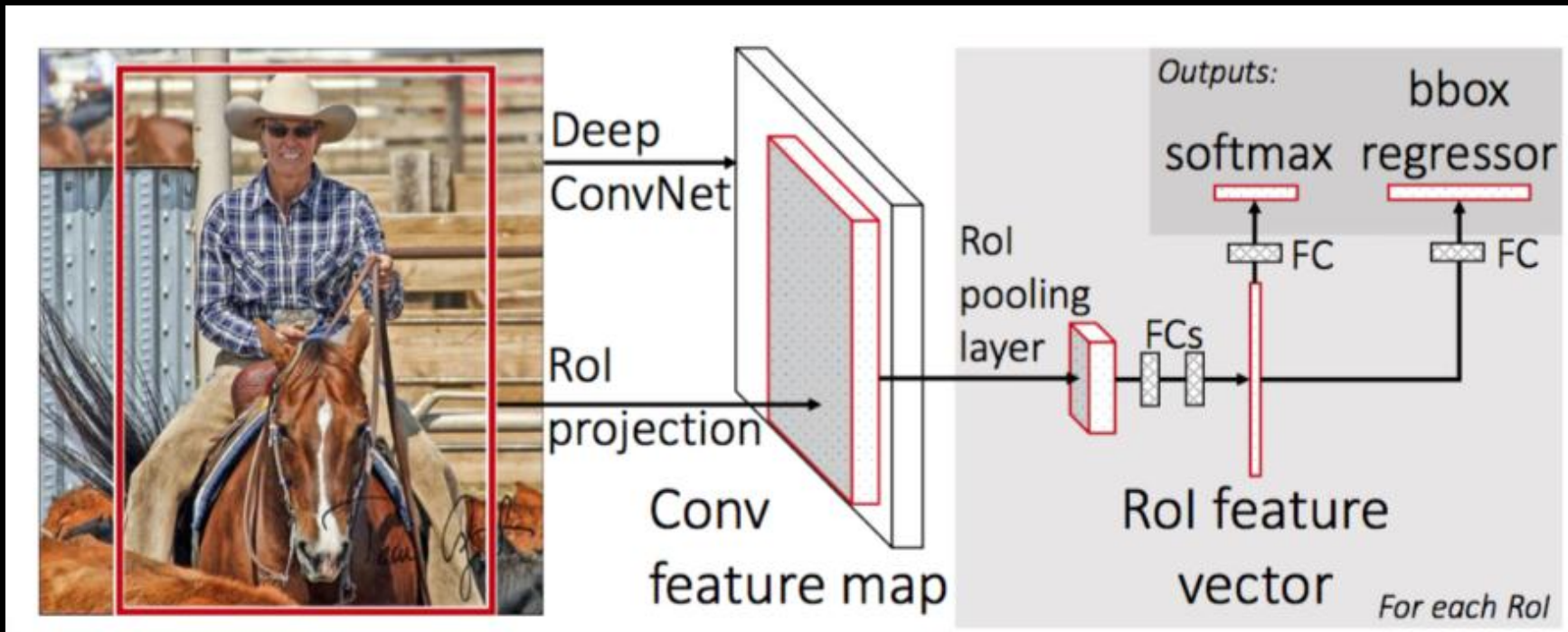


R-CNN - Region-Based Convolutional Neural Networks

1. **Input Image:** Start with a single input image containing one or more objects.
2. **Region Proposal Generation:** Use Selective Search to generate around 2,000 region proposals (potential object locations).
3. **Warp & Feature Extraction:** Each proposed region is cropped and resized (warped) to a fixed size. Then pass each region through a CNN to extract feature vectors.
4. **Region Classification:** Use the extracted features to classify each region using SVMs into object categories (e.g. person, car) or background.

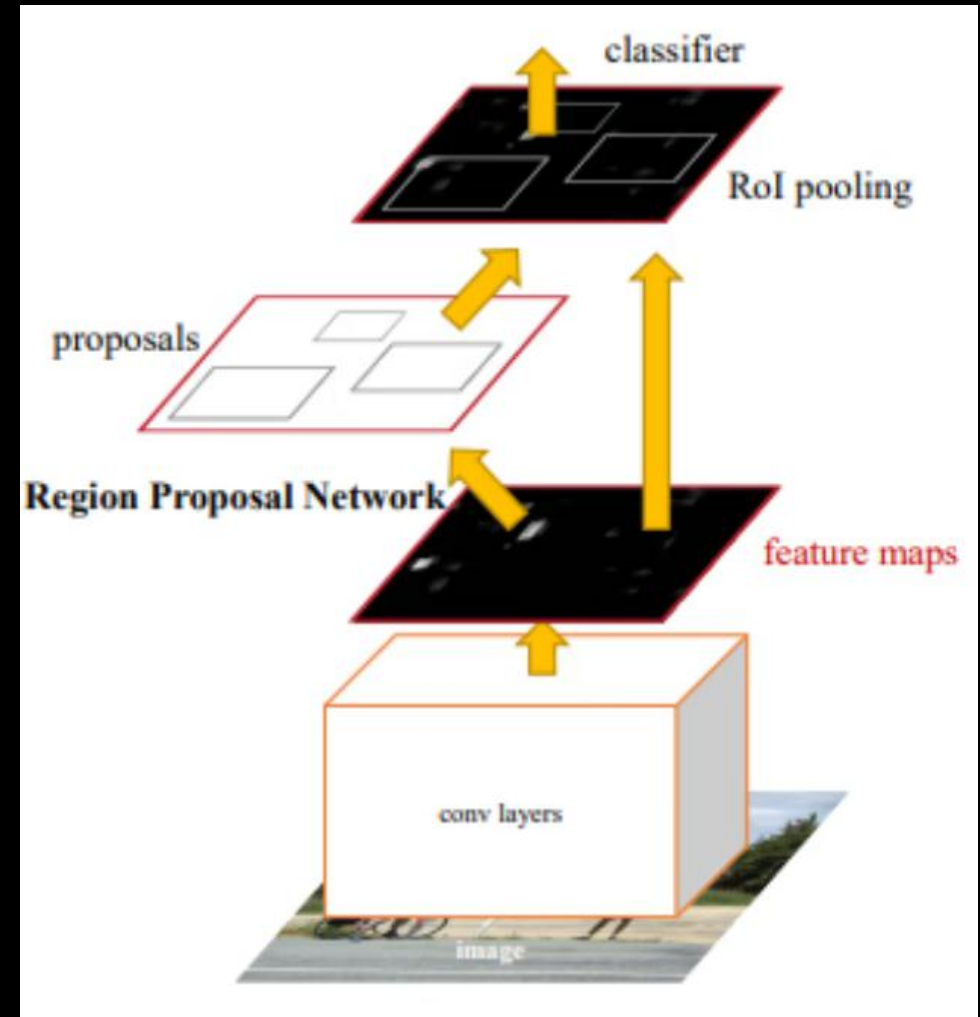
Fast R-CNN

- Fast R-CNN is an improved version of the R-CNN that aggregates CNN features independent of their region of interest (ROI) into a single forward pass over the image. Generally, R-CNN (Region selection with CNN features) is slow because it performs a ConvNet forward pass for each object proposal, without sharing computation.



Faster R-CNN

Faster R-CNN is an object detection model that improves on Fast R-CNN by utilizing a region proposal network (RPN) with the generated feature maps from the convolutional layer, to estimate a region-based object classification (ROI pooling).



Mask R-CNN

- The Mask R-CNN is a typical Object Instance Segmentation technique for object detection.
- This architecture is an extension of Faster R-CNN by adding a branch for predicting segmentation masks on each RoI, in parallel with the existing branch for classification and bounding box regression.
- The mask branch is a small FCN applied to each RoI, predicting a segmentation mask in a pixel-to-pixel manner. Below is an architectural demonstration of Mask R-CNN.

